
Universidad Panamericana

Facultad de Ingeniería · Ingeniería Mecatrónica · 6.º semestre

Reporte Final de Proyecto

Sistema IoT de Control y Monitoreo de Motor DC

Materia de Verano: Internet of Things (IoT)

Alumno 1: Eduardo Hernández Torres

Alumno 2: Carlo Gordillo Arenas

Alumno 3: Alejandro Acosta Salazar

Equipo #: 2

Tema especializado: Control PID

Profesor: Zyanya Alejandra Orozco Olivares

Fecha: July 29, 2026

GitHub: <https://github.com/Eduardoht1/IoT/tree/main/Proyecto/>

Tabla de calificación del reporte

(uso exclusivo del profesor)

Sección	Obj.	Marco	Arq.	Cód.	Result.	Anál.	Esp.	Concl.	Total
Puntos	/5	/10	/10	/10	/15	/15	/25	/10	/100

Contents

1	Objetivo del proyecto	3
2	Marco teórico	3
2.1	Arquitectura IoT de 4 capas	3
2.2	Protocolos de comunicación inalámbrica	3
2.3	Control PID — fundamentos y sintonización	3
2.4	Pipeline de datos: MQTT → Telegraf → InfluxDB	4
2.5	Visualización y monitoreo: Grafana e HTML	4
3	Arquitectura del sistema	5
3.1	Diagrama de bloques	5
3.2	Descripción de cada componente	6
3.3	Tabla de conexiones físicas del hardware	7
4	Código fuente	8
4.1	ESP32 — PID + publicación MQTT (main.cpp)	8
4.2	Parámetros del sistema	12
4.3	Consultas Flux del dashboard de Grafana	12
5	Resultados	14
5.1	Capturas de pantalla del sistema	14
5.1.1	Serial Monitor del ESP32	14
5.1.2	Dashboard de Grafana con datos reales	14
5.1.3	Dashboard HTML personalizado	15
5.2	Experimento de escalón — respuesta del PID	15
6	Análisis y discusión	16
6.1	¿El PID logra error cero en estado estacionario?	16
6.2	¿Qué término del PID fue el más crítico para tu motor?	16
6.3	Latencia del pipeline: ¿cuánto tarda un dato del ESP32 en aparecer en Grafana?	16
6.4	¿Qué pasaría si el WiFi falla mientras el motor está girando?	16
6.5	¿Cuáles son las limitaciones del sistema que identificaron?	16
7	Sección especializada del equipo	18
8	Conclusiones	18
9	Trabajo futuro	18

1. Objetivo del proyecto

Instrucción: Describe en 3–4 oraciones el sistema completo que construyeron. No copies el enunciado del problema — explica qué hace el sistema, para qué sirve y qué valor tiene en un contexto industrial real.

Lo que se construyó en este proyecto fue un sistema de control de velocidad en lazo cerrado con un motor DC, que lleva un algoritmo PID en un microcontrolador ESP32, para transmitir datos a través de una red de WiFi, trabajada por el equipo.

2. Marco teórico

Instrucción: Explica los conceptos clave del sistema completo. Mínimo 5 subsecciones. Escribe con tus propias palabras — conecta cada concepto con el proyecto.

2.1. Arquitectura IoT de 4 capas

Guía: ¿Cómo encaja tu sistema en las 4 capas (percepción, red, middleware, aplicación)? Da un ejemplo concreto de cada capa con un componente de tu proyecto.

En este proyecto, la capa Física abarca las mediciones del sensor Hall y las acciones físicas del motor DC; la capa de Red transporta estos datos a través del WiFi usando el protocolo MQTT; la capa de Middleware procesa y almacena esta información mediante el broker Mosquitto y la base de datos InfluxDB; y, por último, la capa de Aplicación se materializa en el dashboard web HTML y Grafana, el cual permite al usuario visualizar las gráficas en tiempo real y enviar comandos de control

2.2. Protocolos de comunicación inalámbrica

Guía: ¿Qué protocolo eligieron para comunicar el motor con el servidor? Justifica la elección frente a las alternativas (BLE, WiFi, LoRa). Menciona alcance, consumo y velocidad.

Utilizamos el protocolo de WIFI, por su velocidad inmediata y rápida respuesta que necesitamos en nuestro motor para que sea controlado desde una distancia considerable.

2.3. Control PID — fundamentos y sintonización

Guía: Explica qué hacen los términos P, I y D. ¿Cómo los sintonizaron? ¿Qué método usaron (Ziegler-Nichols, función de transferencia, empírico)?

El Controlador Proporcional Integral Derivativo es un algoritmo que ayuda a calcular los errores $e(t)$ entre un setpoint deseado y la velocidad real del motor para ajustar la salida de la señal PWM.

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

Se divide en 3 partes: [Proporcional K_p]: Reacciona al error actual y le da un empujón principal al motor. [Integral K_i]: Suma errores pasados para quitar la diferencia que falta para llegar a una velocidad esperada y correcta. [Derivativo K_d]: Predice un cambio futuro para evitar que el motor acelere de golpe o tenga jalones. Los valores que se ajustan a prueba y error en el laboratorio, desde K_p para dar velocidad, K_i corrige imprecisiones y K_d estabiliza las respuestas.

2.4. Pipeline de datos: MQTT → Telegraf → InfluxDB

Guía: Explica cada pieza del pipeline y por qué el ESP32 publica a MQTT en lugar de escribir directo a InfluxDB. Incluye el concepto de desacoplamiento.

El camino de los datos desde el ESP32 hasta guardarse en base de datos. Estos datos tienen un viaje de 3 etapas: Primero se manda la velocidad en MQTT a mosquitto, luego Telegraf toma esos datos y los acomoda y finalmente se guardan en hora exacta en InfluxDB. El ESP32 trabaja más rápido sin saturar enviando datos pesados por internet, y si la base de datos se apaga o falla el control del motor sigue funcionando sin detenerse y el MQTT guarda en la base de datos para desacoplar el sistema.

2.5. Visualización y monitoreo: Grafana e HTML

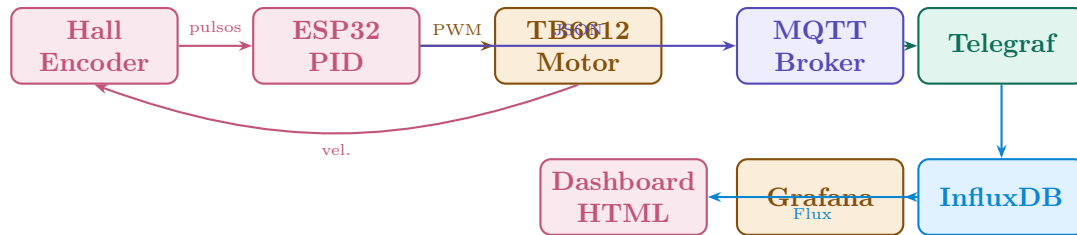
Guía: ¿Qué diferencia hay entre el dashboard de Grafana y el dashboard HTML? ¿Cuál tiene mayor frecuencia de actualización y por qué?

Son las dos pantallas existentes del proyecto y o dos porque tienen la función de controlar el programa, actualizarse desde MQTT de forma instantánea, en vivo desde una página web HTML. Mientras la otra pantalla tiene el análisis histórico en Grafana, que consulta la base de datos del InfluxDB en poco lapso de tiempo para mostrar en gráficas el comportamiento del motor.

3. Arquitectura del sistema

Instrucción: Presenta el diagrama de bloques completo del sistema. Indica qué dato fluye en cada flecha y en qué dirección. Puedes usar el diagrama TikZ de abajo como base o insertar una imagen.

3.1. Diagrama de bloques



3.2. Descripción de cada componente

Componente	Función en el sistema	Datos que envía / recibe
Sensor Hall	Detecta el paso de los imanes del encoder del motor para medir la velocidad de rotación.	Envía: Pulsos eléctricos digitales a la ESP
ESP32 + PID	Lee la velocidad del sensor, ejecuta el algoritmo de control PID en tiempo real, genera la señal PWM y publica telemetría por WiFi.	Envía: Señales PWM y datos de telemetría en formato JSON por MQTT.
TB6612FNG	Puente H que amplifica la señal de control de la ESP para conmutar la potencia necesaria para el motor DC.	Recibe: Pulsos del Sensor Hall y comandos de setpoint/control desde MQTT.
Mosquitto (MQTT)	Centraliza la comunicación entre el ESP32, Telegraf y la interfaz HTML.	Recibe: Señales de control lógico y PWM desde el ESP32.
Telegraf	Agente que se suscribe al broker MQTT, procesa los JSONs recibidos y los transforma al Line Protocol de InfluxDB.	Envía: Voltaje variable hacia el motor DC.
InfluxDB	Base de datos de series de tiempo que almacena y organiza los datos de telemetría con sus respectivos timestamps.	Recibe: Telemetría y comandos de setpoint.
Grafana	Herramienta de visualización que consulta InfluxDB mediante Flux para renderizar gráficos e indicadores en tiempo real y gestionar alertas.	Envía: Redistribuye los mensajes a los clientes suscritos (Telegraf y Dashboard HTML).
Dashboard HTML	Interfaz web de usuario final para monitorear el sistema e ingresar un valor de setpoint para el motor.	Recibe: Mensajes JSON desde Mosquitto mediante suscripción MQTT.

3.3. Tabla de conexiones físicas del hardware

Pin	GPIO ESP32	Descripción
PWMA (TB6612)	13 o 26 (mediante software)	Señal PWM de velocidad
AIN1 (TB6612)	GPIO 26	Dirección bit 1
AIN2 (TB6612)	GPIO 13	Dirección bit 2
Hall signal	GPIO 27 (Canal A) y GPIO 4 (Canal B)	Pulsos del encoder

4. Código fuente

Instrucción: Pega tu código real — el que flasheaste al ESP32 y el que corre en la laptop. NO el código de ejemplo de las guías. Incluye comentarios que expliquen las partes más importantes.

4.1. ESP32 — PID + publicación MQTT (main.cpp)

Listing 1: main.cpp – código final del ESP32

```

1 // Motor DC con PID + publicacion MQTT a InfluxDB/Grafana
2 // Universidad Panamericana - IoT Semana 2
3
4 #include <Arduino.h>
5 #include <WiFi.h>
6 #include <PubSubClient.h>
7 #include <ArduinoJson.h>
8
9 // WiFi
10 const char* WIFI_SSID      = "LaptopEdu";
11 const char* WIFI_PASSWORD = "G65n72R98a02E05";
12
13 // MQTT (IP de la laptop con Docker)
14 const char* MQTT_BROKER = "192.168.137.1";
15 const int MQTT_PORT = 1883;
16
17 const char* TOPIC_TELEMETRIA = "iot/motor/telemetry";
18 const char* TOPIC_SETPOINT   = "iot/motor/setpoint";
19 const char* TOPIC_DIRECCION  = "iot/motor/direccion"; // <-- Nuevo t pico
20
21 // Gpios
22 #define AIN2 13 // Usado como PWM/Dirección A
23 #define AIN1 26 // Usado como PWM/Dirección B
24 #define Amarillo 27 // Encoder Canal A
25 #define Azul 4 // Encoder Canal B
26
27 // Relación de transmisión y encoder
28 const float PPR = 11.0;
29 const float GEAR_RATIO = 35.0;
30 const float REAL_PPR = PPR * GEAR_RATIO;
31
32 // Constantes del Controlador PI
33 float Kp = 1.5;
34 float Ki = 12.5;
35
36 // Variables del PID
37 float setpoint = 70.0;
38 float rpm = 0.0;
39 float integral = 0.0;
40 float pwmSalida = 0.0;
41 int direccion = 1; // 1 = CW, 0 = CCW <-- Variable de estado direccional
42
43 // Variables del sensor Hall
44 volatile long encoderPulses = 0;
45 portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
46
47 // Temporizadores

```

```

48 const int interval = 20; // 20 ms (50 Hz)
49 const float dt = interval / 1000.0;
50 unsigned long previousMillis = 0;
51
52 #define T_MQTT_MS 500
53 uint32_t tMQTT = 0;
54
55 // MQTT
56 WiFiClient wifiClient;
57 PubSubClient mqttClient(wifiClient);
58 bool mqttListo = false;
59
60 // ISR: cuenta pulsos
61 void IRAM_ATTR countPulses() {
62     portENTER_CRITICAL_ISR(&mux);
63     if (digitalRead(Azul) > 0) {
64         encoderPulses++;
65     } else {
66         encoderPulses--;
67     }
68     portEXIT_CRITICAL_ISR(&mux);
69 }
70
71 // Lógica de control unificada
72 void ejecutarCicloPID() {
73     long currentPulses;
74     portENTER_CRITICAL(&mux);
75     currentPulses = encoderPulses;
76     encoderPulses = 0;
77     portEXIT_CRITICAL(&mux);
78
79     rpm = abs(((float)currentPulses / REAL_PPR) * (60.0 / dt));
80     float error = setpoint - rpm;
81
82     integral += error * dt;
83     float max_integral = 255.0 / Ki;
84     if (integral > max_integral) integral = max_integral;
85     if (integral < -max_integral) integral = -max_integral;
86
87     float control_signal = (Kp * error) + (Ki * integral);
88     int pwm_output = (int)control_signal;
89
90     if (pwm_output > 255) pwm_output = 255;
91     if (pwm_output < 0) pwm_output = 0;
92
93     // Lógica de puente H bidireccional
94     if (direccion == 1) {
95         analogWrite(AIN1, 0); // Apaga canal contrario
96         analogWrite(AIN2, pwm_output); // Inyecta PWM
97     } else {
98         analogWrite(AIN2, 0);
99         analogWrite(AIN1, pwm_output);
100     }
101
102     pwmSalida = (float)pwm_output;

```

```

103 }
104
105 void conectarWiFi() {
106     Serial.printf("Conectando a WiFi '%s'...", WIFI_SSID);
107     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
108     int n = 0;
109     while (WiFi.status() != WL_CONNECTED && n++ < 40) {
110         delay(500); Serial.print(".");
111     }
112     if (WiFi.status() == WL_CONNECTED) {
113         Serial.printf("\n OK IP: %s\n", WiFi.localIP().toString().c_str());
114     }
115 }
116
117 // Callback MQTT: recibe setpoint o direcci n
118 void mqttCallback(char* topic, byte* payload, unsigned int len) {
119     String msg = "";
120     for (unsigned int i = 0; i < len; i++) msg += (char)payload[i];
121
122     if (String(topic) == TOPIC_SETPOINT) {
123         float sp = msg.toFloat();
124         if (sp >= 0 && sp <= 200) { // Limitado al rango de la gr fica
125             setpoint = sp;
126             integral = 0; // Reset windup
127             Serial.printf("<-- Nuevo setpoint: %.1f RPM\n", setpoint);
128         }
129     }
130     else if (String(topic) == TOPIC_DIRECCION) {
131         int dir = msg.toInt();
132         if (dir == 0 || dir == 1) {
133             direccion = dir;
134             integral = 0; // Reset por cambio brusco de fase
135             Serial.printf("<-- Nueva direcci n: %s\n", direccion == 1 ? "CW" :
136                 "CCW");
137         }
138     }
139
140 void conectarMQTT() {
141     if (WiFi.status() != WL_CONNECTED) return;
142     int n = 0;
143     while (!mqttClient.connected() && n++ < 5) {
144         Serial.print("Conectando MQTT...");
145         if (mqttClient.connect("ESP32-Motor-PID")) {
146             mqttClient.subscribe(TOPIC_SETPOINT);
147             mqttClient.subscribe(TOPIC_DIRECCION); // Suscripci n a direcci n
148             mqttListo = true;
149             Serial.println(" OK");
150         } else {
151             delay(1000);
152         }
153     }
154 }
155
156 void publicarTelemetry() {

```

```

157  if (!mqttListo || !mqttClient.connected()) return;
158
159  StaticJsonDocument<256> doc;
160  float error_val = setpoint - rpm;
161
162  doc["rpm"] = round(rpm * 10.0f) / 10.0f;
163  doc["setpoint"] = setpoint;
164  doc["error"] = round(error_val * 10.0f) / 10.0f;
165  doc["pwm"] = round(pwmSalida);
166  doc["pwm_pct"] = round((pwmSalida / 255.0) * 100.0); // % de Duty Cycle
167
168  // % Error relativo (Manejo de div / 0)
169  doc["error_pct"] = setpoint > 0 ? round((abs(error_val) / setpoint) *
    100.0) : 0;
170  doc["direccion"] = direccion;
171  doc["nodo"] = "ESP32-Motor";
172
173  char buf[256];
174  serializeJson(doc, buf);
175  mqttClient.publish(TOPIC_TELEMETRIA, buf);
176 }
177
178 void setup() {
179     Serial.begin(115200);
180
181     pinMode(AIN1, OUTPUT);
182     pinMode(AIN2, OUTPUT);
183     digitalWrite(AIN1, LOW);
184     digitalWrite(AIN2, LOW);
185
186     pinMode(Amarillo, INPUT_PULLUP);
187     pinMode(Azul, INPUT_PULLUP);
188     attachInterrupt(digitalPinToInterrupt(Amarillo), countPulses, RISING);
189
190     conectarWiFi();
191     if (WiFi.status() == WL_CONNECTED) {
192         mqttClient.setServer(MQTT_BROKER, MQTT_PORT);
193         mqttClient.setCallback(mqttCallback);
194         conectarMQTT();
195     }
196
197     previousMillis = millis();
198     tMQTT = millis();
199 }
200
201 void loop() {
202     if (mqttListo) mqttClient.loop();
203
204     unsigned long currentMillis = millis();
205
206     // Loop PI de control en tiempo discreto (20ms)
207     if (currentMillis - previousMillis >= interval) {
208         ejecutarCicloPID();
209         previousMillis = currentMillis;
210     }

```

```

211 if ((currentMillis - tMQTT) >= T_MQTT_MS) {
212     publicarTelemetry();
213     tMQTT = currentMillis;
214     if (mqttListo && !mqttClient.connected()) conectarMQTT();
215 }
216 }
217 }

```

4.2. Parámetros del sistema

Parámetro	Valor	Cómo lo determinaron
K_p	1.5	Para la obtención de estos valores se hizo una sintonización asistida por software (Matlab).
K_i	12.5	Para la obtención de estos valores se hizo una sintonización asistida por software (Matlab).
K_d	0.0 (No utilizado)	Para la obtención de estos valores se hizo una sintonización asistida por software (Matlab). En este caso decidimos no usar la derivada para simplificar el control.
Intervalo PID (ms)	20 ms (50 Hz)	Definido por el tiempo de muestreo discreto $\text{interval} = 20$ en el temporizador del <code>loop()</code> para asegurar una constante $\text{dt} = 0.02$ estable.
Intervalo MQTT (ms)	500 ms (2 Hz)	Definido con <code>define T_MQTT_MS 500</code> para evitar saturar el ancho de banda del broker y no ralentizar el bucle de control crítico de 20 ms.
PPR sensor Hall	385 PPR	Multiplicando los pulsos base del encoder ($\text{PPR} = 11.0$) por la relación de la caja reductora del motor ($\text{GEAR_RATIO} = 35.0$).
Protocolo elegido	Wi-Fi + MQTT	Se eligió Wi-Fi por sus altas capacidades de transmisión de datos a distancia moderada sin cuellos de botella, y por la facilidad con la que permite desplegar todo un ecosistema IoT escalable (conectando nativamente el ESP32 con el broker MQTT local, Docker, InfluxDB y Grafana).

Table 1: Parámetros del sistema de control

4.3. Consultas Flux del dashboard de Grafana

Listing 2: Consulta principal — RPM y Setpoint

```
from(bucket: "iot-verano")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_field"] == "pwm")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "pwm_control")
```

5. Resultados

Instrucción: Presenta evidencia visual del sistema funcionando. Mínimo 3 capturas de pantalla. Cada imagen debe tener un título descriptivo y un párrafo explicando qué muestra.

5.1. Capturas de pantalla del sistema

5.1.1. Serial Monitor del ESP32

```
Conectando a WiFi 'LaptopEdu'....  
OK IP: 192.168.137.82  
Conectando MQTT... OK  
Sistema listo  
Setpoint inicial: 70.0 RPM  
<-- Nuevo setpoint: 110.0 RPM  
<-- Nuevo setpoint: 40.0 RPM
```

Figure 1: Serial Monitor

5.1.2. Dashboard de Grafana con datos reales



Figure 2: Dashboard Grafana

5.1.3. Dashboard HTML personalizado



Figure 3: Dashboard Grafana

5.2. Experimento de escalón — respuesta del PID

Instrucción: Llena la tabla con los datos reales de tu motor. Espera al menos 20 segundos en cada setpoint antes de anotar.

Setpoint (RPM)	RPM estado est.	Error residual	PWM est.	Observación
200	40	31 - 47	-7 - 9	46 - 70
400	80	70 - 85	-5 - 10	71 - 105
600	120	109 - 133	-13 - 11	133 - 164
300	170	140 - 164	6 - 30	255

Table 2: Respuesta del PID a distintos setpoints

6. Análisis y discusión

Instrucción: No repitas los resultados — interprétalos. Cada pregunta debe responderse con fundamento técnico y datos del experimento.

6.1. ¿El PID logra error cero en estado estacionario?

Llega en momentos, pero nunca se estabiliza (por el funcionamiento del encoder y el ruido que este tiene), pero si se llegara a usar un filtro ema (Exponential Moving Average o Promedio Móvil Exponencial), si se llegaria a ver por mayor tiempo.

6.2. ¿Qué término del PID fue el más crítico para tu motor?

En nuestro caso el mas importante es el ki, debido a que de este depende el error de estado estacionario.

6.3. Latencia del pipeline: ¿cuánto tarda un dato del ESP32 en aparecer en Grafana?

Etapa	Latencia estimada	Justificación
ESP32 → MQTT	1 - 5 ms	Publicación en red Wi-Fi local con el protocolo MQTT
MQTT → Telegraf	1 - 2 ms	Suscripción interna en la red de Docker; Mosquitto notifica inmediatamente a Telegraf por eventos.
Telegraf → InfluxDB	10 - 50 ms	Procesamiento del JSON a Line Protocol
InfluxDB → Grafana	0.5 - 5 s	epende del intervalo de refresco configurado en el dashboard (ej. 5s) más el tiempo de ejecución de la consulta Flux.
Total percibido	0.5 - 5 s	Formado casi en su totalidad por el intervalo de refresco automático (auto-refresh) configurado en el dashboard de Grafana.

6.4. ¿Qué pasaría si el WiFi falla mientras el motor está girando?

El sistema seguiría funcionando con el setpoint seleccionado sin ningún tipo de problema, únicamente se perdería la información histórica de lo que suceda mientras no haya conexión.

6.5. ¿Cuáles son las limitaciones del sistema que identificaron?

A pesar del buen desempeño general de la telemetría, el protocolo de comunicación inalámbrico introduce una latencia variable, lo cual nos limita el uso del dashboard remoto para realizar un

control en lazo cerrado en tiempo real, reservando la capa IoT principalmente para monitoreo, cambio de setpoints y supervisión.

7. Sección especializada del equipo

★ SECCIÓN ESPECIALIZADA DE TU EQUIPO:

Esta sección cubre el tema que le tocó a tu equipo en la presentación final. Es la sección más profunda del reporte — desarrolla tu tema con mayor detalle técnico que el resto de los equipos.

Tu tema: Control PID

8. Conclusiones

Instrucción: Mínimo **4 conclusiones numeradas**. Cada una debe ser verificable con datos reales del proyecto, no una opinión general. Al menos una debe referirse específicamente al tema especializado de su equipo.

1. La evaluación comparativa determinó que el protocolo Wi-Fi cumplió con los requerimientos de transmisión al registrar un enlace estable durante las pruebas dinámicas. Esta configuración garantizó el balance óptimo entre bajo consumo energético, rendimiento y la tasa de refresco requerida para la transmisión continua de datos del motor.
2. A partir de la función de transferencia identificada, el control PI sintonizado con parámetros $K_p = 1.5$ y $K_i = 12.5$ logró eliminar el sobrepico del 6% presente en la respuesta base a 0%. Asimismo, redujo el settling time de 0.6 segundos a aproximadamente 0.2 segundos, asegurando una respuesta sobrearmortiguada, rápida y estable sin oscilaciones destructivas.
3. La arquitectura de datos desacoplada permitió procesar y almacenar de manera confiable las lecturas de velocidad del motor (muestreadas en intervalos continuos de 20 segundos). La estructuración mediante Tags para metadatos del dispositivo y Fields para las series temporales de amplitud optimizó las consultas en InfluxDB y garantizó cero pérdida de información en el broker.
4. Se logró la visualización unificada del sistema integrando Grafana para el análisis histórico y un dashboard web HTML/JS para el monitoreo en tiempo real. La interfaz respondió de manera fluida manteniendo una sincronización constante con el backend.

9. Trabajo futuro

Instrucción: Si tuvieran una semana más, ¿qué agregarían o mejorarían? Séan específicos — no “mejorar el PID” sino “implementar un observador de estado para estimar la velocidad sin encoder”.

Implementación de retención de datos con downsampling en InfluxDB mediante Tasks: Para evitar el almacenamiento de datos de forma masiva, nos gustaría implementar un downsampling que nos ayude a reducir en gran medida la cantidad de datos.

Referencias

Instrucción: Mínimo **5 referencias**. Incluir: datasheet del TB6612FNG, documentación de InfluxDB, documentación de la librería MQTT usada, y al menos 2 fuentes adicionales relevantes a su tema especializado.

References

- [1] Toshiba Bi-CD Integrated Circuit Silicon Monolithic TB6612FNG Driver IC for Dual DC motor. (n.d.).
- [2] InfluxData. (2026). Influxdata.com. <https://docs.influxdata.com/>
- [3] OASIS. *MQTT Version 5.0 Specification*. 2019.
- [4] Espressif Systems. *ESP32 Technical Reference Manual*. 2024.
- [5] JGA25-371 DC Gearmotor with Encoder (126 RPM at 12 V). (n.d.). https://people.ece.ubc.ca/~eng-services/files/courses/elec391-data_sheets/MOT4-info.pdf

Lista de verificación antes de entregar:

- | | |
|---|---|
| ☒ Portada completa (nombres, tema, GitHub) | datos reales |
| ☒ Diagrama de bloques completo con flujo de datos | ☒ Tabla de latencia del pipeline completa |
| ☒ Tabla de conexiones físicas completa | ☒ Sección especializada desarrollada en profundidad |
| ☒ Código real pegado (no el de ejemplo) | ☒ Mínimo 4 conclusiones con datos reales |
| ☒ Tabla de parámetros PID completa | ☒ Mínimo 5 referencias |
| ☒ 3 capturas de pantalla insertadas | ☒ PDF compilado sin errores |
| ☒ Tabla del experimento de escalón con | ☒ <code>.tex</code> subido a GitHub |